

AI FitTrack API:

AI-Powered Fitness Tracking and Recommendation Backend

Team ID	UNM1611BHAVANA
Project Name	AI FitTrack API
Date	08 October 2026
Team Members	BHAVANA, THEJASRI, TAMILVANA, VASIGARAN

Project Description:

AI FitTrack API is a RESTful backend that combines secure user management, workout tracking and Generative AI to give users personalised fitness guidance. The platform includes an Authentication module (register, login and profile with JWT-based access), a Workout module for logging and managing workout records, and an AI module that offers two features: a Workout Recommendation generator and a Fitness Insights generator that analyses a user's totals, average duration and calories burned.

Built with Node.js and Express, backed by MongoDB (Mongoose) and integrated with a generative AI service through a dedicated service layer (geminiService.js), the API follows a clean routes → controllers → services → models structure. Input validation, protected routes and secure environment-variable management (.env) keep the system reliable and safe, and the API can be tested end-to-end with Postman using the included FitTrack.postman_collection.json.

Scenarios

Scenario 1: A new user registers with a name, email and password. The API stores the account with a hashed password and returns a success response with a signed JWT token and the user's id, so the user can immediately access protected endpoints.

Scenario 2: A logged-in user requests a workout recommendation. The AI module builds a prompt from the user's details and goals, calls the AI service, and returns a structured personalised workout plan.

Scenario 3: A user submits fitness statistics (totalWorkouts, averageDuration, totalCaloriesBurned) to the fitness-insights endpoint. The API validates the input and the AI returns insights and suggestions; if any required field is missing, it responds with a clear 400 Bad Request error message.

Technical Architecture:

Layer	Description
Client	Postman / any API client (or a future front end) sends HTTP requests with JSON bodies.
Express Server	server.js starts the app and connects to MongoDB; app.js registers middleware and mounts routes under /api.
Routes	authRoutes, workoutRoutes and aiRoutes define the REST endpoints; index.js combines them.
Controllers	authController, workoutController and aiController hold request handling and

Layer	Description
	business logic, including validation.
Middleware	Authentication (JWT verification), validation and centralized error handling.
Services	geminiService.js communicates with the generative AI API and returns the generated content.
Models / Database	Mongoose models stored in MongoDB (local connection).
Configuration	config/ for database settings; .env for secrets (API key, JWT secret, port).

Description: AI FitTrack API uses a modular layered architecture. Requests flow from the client through routes and middleware to the controllers; controllers use models for MongoDB persistence and the Gemini service for AI-generated content. Secrets are never hard-coded and are read from the .env file.

ER Diagram

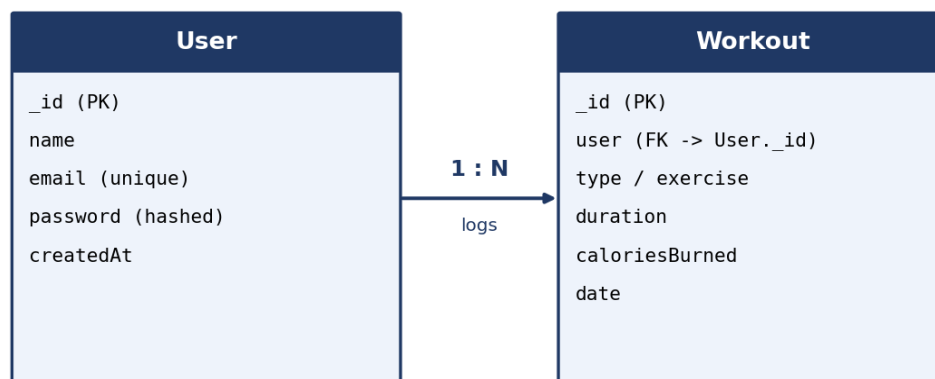


Figure: Entity relationship diagram (User 1 : N Workout)

Description: The database has two main collections. The User collection stores the account details (name, unique email and a hashed password). The Workout collection stores each workout record and references the owning user through a user id, giving a one-to-many relationship: one user can log many workouts.

Pre-requisites:

- **Node.js and npm:** runtime and package manager for the backend.
- **Express.js Framework Knowledge:** routing, middleware and controllers.
- **MongoDB and Mongoose:** database setup and schema modelling (MongoDB running locally).
- **REST API and JWT Authentication:** token-based authentication concepts.
- **Generative AI API Familiarity:** API key creation and usage for the Gemini service.
- **Postman:** testing API endpoints.
- **Version Control with Git:** repository management.
- **Visual Studio Code:** development environment.

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate an AI API key and configure it securely in the .env file.
- **Activity 1.2:** Research and select the generative AI model for workout recommendations and fitness insights.
- **Activity 1.3:** Define the architecture of the application (routes, controllers, services, models).
- **Activity 1.4:** Set up the development environment and install dependencies.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the authentication module (register, login, profile) and the workout module.
- **Activity 2.2:** Develop the AI features: workout recommendation and fitness insights.

Milestone 3: Server and API Development

- **Activity 3.1:** Write server.js and app.js, connect MongoDB, mount routes and add error handling.

Milestone 4: Testing the API

- **Activity 4.1:** Test all endpoints with Postman using the included collection.
- **Activity 4.2:** Verify validation and error responses.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment and run it with npm start.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone we select the generative AI model and define the architecture so the API can deliver personalised workout recommendations and fitness insights.

Activity 1.1: Generate an AI API Key

Create an API key from the AI provider's console, copy it immediately and store it only in the .env file in the project root. Never commit .env; add it to .gitignore.

```
GEMINI_API_KEY=your_api_key_here
JWT_SECRET=your_jwt_secret_here
MONGO_URI=mongodb://localhost:27017/fittrack
PORT=5000
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

- **Understand the Requirements:** the AI must produce workout recommendations and insights from numeric fitness data.
- **Explore Documentation:** review available models, limits, speed and pricing.
- **Evaluate Performance:** compare response quality and generation time (a workout recommendation request took about 7.8 seconds in testing).
- **Select the Model:** choose the Gemini model used by geminiService.js for its balance of quality and speed.

Activity 1.3: Define the Architecture of the Application

- **Routes:** /api/auth, /api/workouts and /api/ai.
- **Controllers:** authController.js, workoutController.js, aiController.js.
- **Services:** geminiService.js wraps the AI API call.
- **Models and Middleware:** Mongoose models, JWT authentication and error handling.

Activity 1.4: Set Up the Development Environment

Install Node.js and npm, open the project folder in Visual Studio Code, and install dependencies:

```
PS D:\NM PROJECT FILES\NM - project code files\CODE> npm install
```

Project structure:

```
CODE/
├── src/
│   ├── config/
│   ├── controllers/ (aiController.js, authController.js, workoutController.js)
│   ├── middleware/
│   ├── models/
│   ├── routes/ (aiRoutes.js, authRoutes.js, index.js, workoutRoutes.js)
│   ├── services/ (geminiService.js)
│   ├── app.js
│   └── server.js
├── .env
├── .env.example
├── .gitignore
├── FitTrack.postman_collection.json
└── package.json
```

Milestone 2: Core Functionalities Development

Activity 2.1: Authentication and Workout Modules

- **Register:** POST /api/auth/register creates a user and returns a JWT token.
- **Login:** POST /api/auth/login authenticates the user.
- **Profile:** GET /api/auth/profile returns the logged-in user's details (protected).
- **Workouts:** workoutController.js and workoutRoutes.js manage workout records.

Activity 2.2: AI Features

- **Workout Recommendation:** POST /api/ai/workout-recommendation returns a personalised workout plan.
- **Fitness Insights:** POST /api/ai/fitness-insights accepts totalWorkouts, averageDuration and totalCaloriesBurned and returns AI-generated insights.

Input validation: if any of the three insight fields is missing, the controller returns 400 with the message "Please provide totalWorkouts, averageDuration, and totalCaloriesBurned".

Milestone 3: Server and API Development

Milestone 3 covers the core server logic. server.js loads environment variables, connects to MongoDB and starts the Express app; app.js registers JSON parsing, logging, routes and error handling.

Activity 3.1: Writing the Main Application Logic

- **/api/auth/register (POST):** create account, return success, token and user.
- **/api/auth/login (POST):** validate credentials, return token.
- **/api/auth/profile (GET):** return user profile for a valid token.
- **/api/ai/workout-recommendation (POST):** generate workout plan through geminiService.
- **/api/ai/fitness-insights (POST):** validate stats and generate insights.

Milestone 4: Testing the API

The API is tested with Postman (or the VS Code API client) against `http://localhost:5000`. The terminal log confirms each request and its status code.

Activity 4.1: Run the Server

```
PS D:\NM PROJECT FILES\NM - project code files\CODE> npm start
> ai-fittrack-api@1.0.0 start
> node src/server.js
```

Server running in development mode on port 5000
MongoDB Connected: localhost

Activity 4.2: Test User Registration and Authentication

A POST request with name, email and password creates the user. The API replies 201 Created with success: true, a signed JWT token and the user object, and the terminal logs the request.

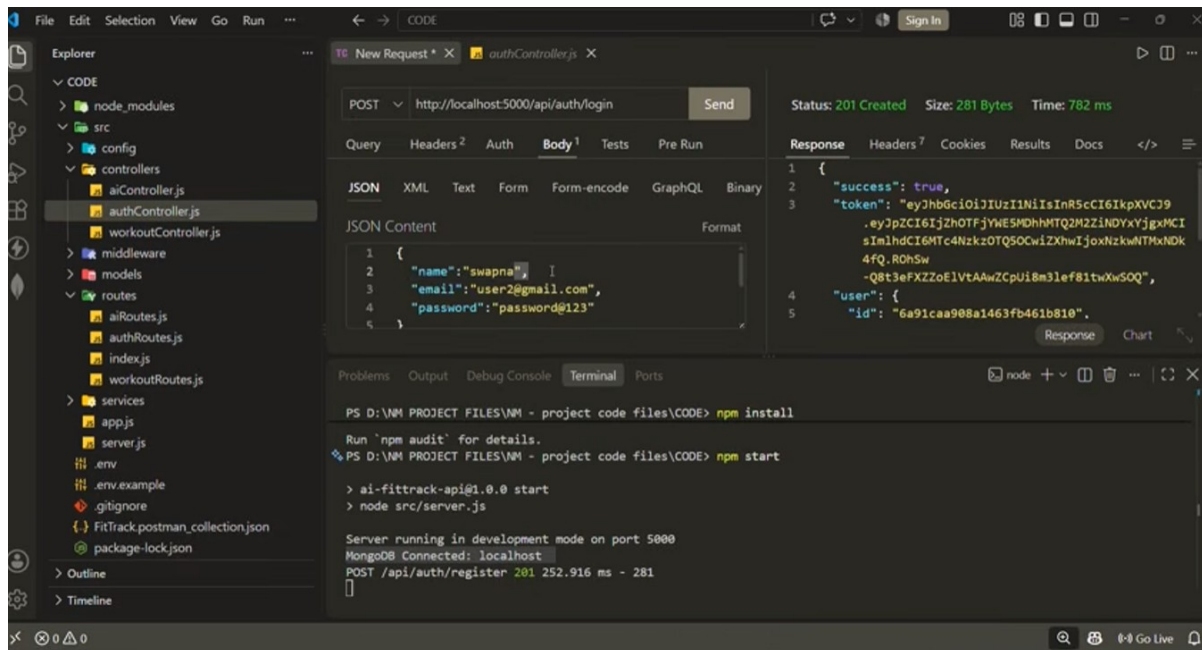


Figure: Authentication request returning 201 Created with JWT token; MongoDB connected and server running on port 5000

Activity 4.3: Test AI Endpoints and Validation

The terminal log shows register (201), login (200), profile (200) and workout-recommendation (200) succeeding. A request to /api/ai/fitness-insights with an incomplete JSON body is correctly rejected with 400 Bad Request and a descriptive error message, proving that input validation works.

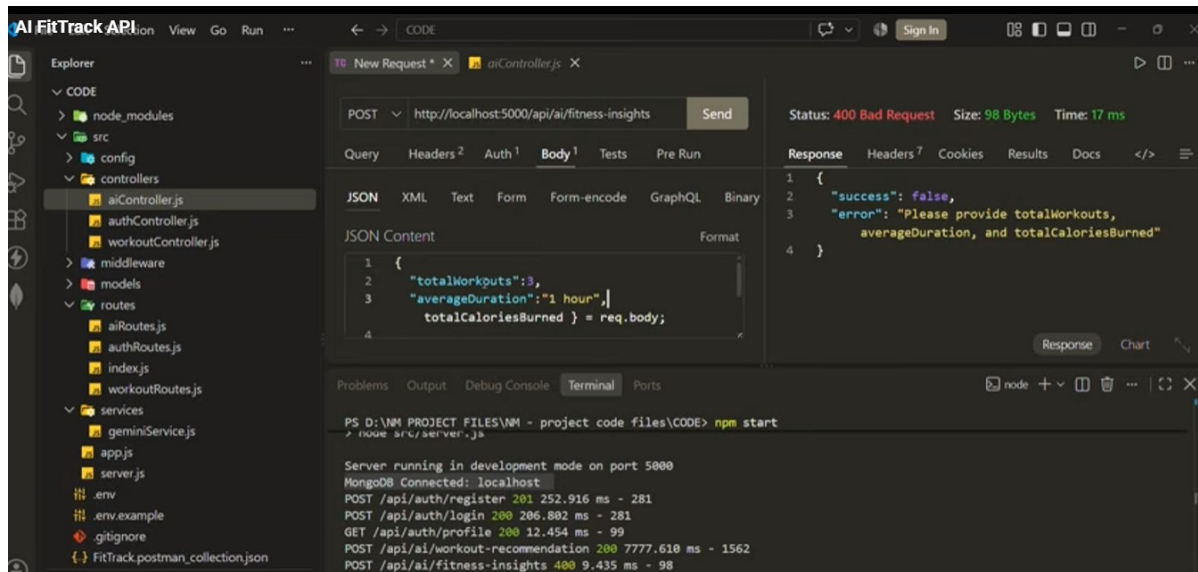


Figure: Fitness-insights endpoint returning 400 Bad Request when required fields are missing; terminal shows earlier successful requests

Milestone 5: Deployment

The application is deployed locally for evaluation.

- **Install:** run npm install in the project root.
- **Configure:** create .env from .env.example and provide the API key, JWT secret and MongoDB connection.
- **Start MongoDB:** make sure the MongoDB service is running locally.
- **Run:** npm start (node src/server.js). The server runs on port 5000.
- **Test:** import FitTrack.postman_collection.json into Postman and run the requests.

Project Executable Files

Date	08 October 2026
Team ID	UNM1611BHAVANA
Project Name	AI FitTrack API
Maximum Marks	3 Marks

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	NA
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	NA

Step 2: File / Folder Structure

Path / File	Purpose
CODE/	Root project directory
src/server.js	Starts the Express server and connects to MongoDB
src/app.js	Registers middleware and routes
src/routes/	REST API route definitions
src/controllers/	Request-handling and business-logic functions
src/models/	MongoDB / Mongoose data models
src/middleware/	Authentication, validation and error handling
src/services/	geminiService.js: generative AI integration
src/config/	Database and application configuration
FitTrack.postman_collection.json	Postman collection for API testing
package.json	Project metadata, scripts and dependencies
.env.example	Example environment-variable configuration

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not deployed / Local evaluation (http://localhost:5000)
Login Credentials (Demo)	Demo credentials can be created through POST /api/auth/register
Platform / Hosting Provider	Local Node.js / Express environment with local MongoDB
Repository Link	To be added - GitHub / GitLab repository URL
Demo Video Link	Not applicable

Step 4: Run Instructions

1. Install Node.js and npm on the system.
2. Open the AI FitTrack API project folder in Visual Studio Code.
3. Open a terminal in the project root directory.
4. Run npm install to install the dependencies listed in package.json.
5. Create a .env file using .env.example as the reference and provide the required values.
6. Make sure the MongoDB service is running and the connection value is correct.
7. Run the application using npm start.
8. Verify that the server starts ("Server running ... on port 5000" and "MongoDB Connected"), then use Postman to access the endpoints.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Repository URL is not included in this reference.	Add the actual GitHub / GitLab URL before final submission.
2	The project is documented as a local API solution; no hosted URL.	Run locally or add the hosted URL when deployed.
3	Environment values depend on the evaluator's local setup.	Configure the required .env values and MongoDB connection before running.

Team Members: BHAVANA, THEJASRI, TAMILVANA, VASIGARAN

Conclusion:

AI FitTrack API is a Node.js, Express and MongoDB backend that brings secure authentication, workout tracking and generative AI together to deliver personalised workout recommendations and fitness insights. Testing in Postman confirmed successful registration with JWT tokens, working profile and AI recommendation endpoints, and correct validation errors for incomplete requests. Looking ahead, the project can be extended with a web or mobile front end, progress charts, wearable-device integration and cloud deployment to become a complete AI fitness companion.